SLOT-D

CSA-0620(DAA)

DHARSHITHA.K

192521093

QUESTION-1

Q1 **Master Theorem: Four Subproblems** 4 marks

Analyze T(n) = 4T(n/2) + n² log n using the Master Theorem. Identify a, b, and f(n), state whether a standard case applies, and justify the asymptotic bound.

Your Answer

Recurrence relation:
T(n)=4T(n/2)+n^2\log n

Comparing with the Master Theorem,
T(n)=aT(n/b)+f(n)

(a=4)
(b=2)
f(n)=n^2\log n

Calculate:
n^{\log_b a}=n^{\log_2 4}=n^2
f(n)=Theta(n^2log n) =Theta(n^{log_b a}log^1 n)

This matches the extended Case 2 of the Master Theorem, where \\(f(n)=\Theta(n^{\log_b a}\log^k n)\\).
T(n)=\Theta(n^2\log^{1+1}n)

Final Answer:
boxed{T(n)=\Theta(n^2\log^2 n)}
The standard basic Master Theorem cases do not directly cover the logarithmic factor in this form, but the extended Case 2 applies.

QUESTION-2

Q2 **Master Theorem: Unequal Growth** 4 marks

Analyze T(n) = 3T(n/2) + n using the Master Theorem, where applicable. Explain which term dominates.

Your Answer

Given recurrence relation:
T(n)=3T(n/2)+n

Comparing with the Master Theorem,
T(n)=aT(n/b)+f(n)
(a=3)
(b=2)
(f(n)=n)

Calculate:
[n^{\log_b a}=n^{\log_2 3}\ approx n^{1.585}]

Since (f(n)=n) grows slower than (n^{\log_2 3}), we have
[f(n)=O(n^{\log_2 3-\epsilon})
for some (epsilon>0).
Therefore, Master Theorem Case 1 applies.
The recursive term n^{log_2 3} dominates the non-recursive term n.
Final Answer:
boxed{T(n)=\Theta(n^{\log_2 3})}

Save Answer Next →

QUESTION-3

Q3 Master Theorem: Square-Root Work — 4 marks

Find the asymptotic complexity of T(n) = 2T(n/4) + √n. State the parameters and justify the applicable case.

Your Answer

```
Given recurrence relation:
T(n)=2T(n/4)+\sqrt{n}

Comparing with the Master Theorem,
T(n)=aT(n/b)+f(n)
a=2
b=4
f(n)=\sqrt{n}=n^{1/2}

Calculate:
n^{\log_b a}=n^{\log_4 2}=n^{1/2}

Since,
 f(n)=\Theta(n^{\log_b a})
Master Theorem Case 2 applies, where the non-recursive work matches the critical term.
T(n)=\Theta(n^{1/2}\log n)
Final Answer:
boxed{T(n)=\Theta(\sqrt{n}\log n)}
```

Save Answer | Next →

QUESTION-4

Q4 Master Theorem: Five Subproblems — 4 marks

A divide-and-conquer algorithm makes 5 recursive calls on subproblems of size n/3 and performs Θ(n²) extra work. Determine its asymptotic time complexity.

Your Answer

```
Given recurrence relation:
T(n)=5T(n/3)+n^2

Comparing with the Master Theorem,
 T(n)=aT(n/b)+f(n)
a=5
b=3
f(n)=n^2

Calculate:
 n^{\log_b a}=n^{\log_3 5}\approx n^{1.465}

Since f(n)=n^2grows faster than n^{\log_3 5},
Master Theorem Case 3 applies. The regularity condition is satisfied because
5(n/3)^2=\frac{5n^2}{9}\leq cn
for some constant c<1
Therefore, the extra work n^2 dominates the recursive work.
Final Answer:
\boxed{T(n)=\Theta(n^2)}
```

Save Answer | Next →

QUESTION-5

Q5 **Master Theorem: Logarithmic Factor** 4 marks

Analyze T(n) = 16T(n/4) + n²/log n. Discuss whether the standard Master Theorem applies directly and provide a justified bound.

Your Answer

Given recurrence relation:
T(n)=16T(n/4)+\frac{n^2}{\log n}
Comparing with the Master Theorem,
T(n)=aT(n/b)+f(n)

a=16
b=4
f(n)=\dfrac{n^2}{\log n}

Calculate:

n^{\log_b a}=n^{\log_4 16}=n^2

Here, f(n)=\Theta(n^2/\log n), which is smaller than n^2by a logarithmic factor.

The standard Master Theorem does not directly cover this case because (f(n) does not fit its basic polynomial-growth cases.

Using a recursion-tree analysis, each level contributes approximately n^2/\log n, and there are \Theta(\log n)levels. The total work is therefore Theta(n^2)

Final Answer:
boxed{T(n)=\Theta(n^2)}

Save Answer Next →

QUESTION-6

Q6 **Substitution: Linearithmic Recurrence** 4 marks

Use the substitution method to prove an asymptotic bound for T(n) = 2T(n/2) + n. State an induction hypothesis and base case.

Your Answer

Given recurrence relation:
T(n)=2T(n/2)+n
1Induction Hypothesis:
Assume that T(n)\leq cn\log_2 n+dn where c,d>0are constants.

2 Base Case:
For n=1 let T(1)=k where k is a constant. Choose d\geq k, so the hypothesis holds.

3\. Inductive Step:
Assume the hypothesis holds for n/2
T(n/2)\leq c\frac n2\log_2(n/2)+d\frac n2

Substituting into the recurrence:
T(n)\leq 2\left[c\frac n2\log_2(n/2)+d\frac n2\right]+n
T(n)\leq cn(\log_2 n-1)+dn+n
T(n)\leq cn\log_2 n+(d+1-c)n

Choose (c\geq1). Then,
T(n)\leq cn\log_2 n+dn

Thus, the induction hypothesis holds.

Final Answer:
\\[\boxed{T(n)=\Theta(n\log n)} \\]

Save Answer Next →

QUESTION-7

Q7 **Substitution: Decreasing by One** 4 marks

Use substitution or expansion to establish a tight asymptotic bound for $T(n) = T(n-1) + n$, assuming $T(1)=1$.

Your Answer

Given recurrence relation:
$T(n) = T(n - 1) + n$, where $T(1) = 1$

Step 1: Expand the recurrence
$T(n) = T(n - 1) + n$
$T(n) = T(n - 2) + (n - 1) + n$
$T(n) = T(n - 3) + (n - 2) + (n - 1) + n$

Continuing until the base case:
$T(n) = T(1) + 2 + 3 + \dots + n$

Since $T(1) = 1$,
$T(n) = 1 + 2 + 3 + \dots + n$

Step 2: Apply the summation formula
$T(n) = n(n + 1) / 2$
$T(n) = (n^2 + n) / 2$

Step 3: Determine the asymptotic bound
The dominant term is n^2.
Therefore, $T(n) = \Theta(n^2)$.

Save Answer Next →

QUESTION-8

Q8 **Substitution: Halving with Constant Work** 4 marks

Prove that $T(n) = T(n/2) + 1$ is $\Theta(\log n)$, assuming n is a power of two.

Your Answer

Given recurrence relation:
$T(n) = T(n / 2) + 1$, where n is a power of two.

Step 1: Expand the recurrence
$T(n) = T(n / 2) + 1$
$T(n) = T(n / 4) + 2$
$T(n) = T(n / 8) + 3$

After k steps,
$T(n) = T(n / 2^k) + k$

Step 2: Find the base case
When $n / 2^k = 1$,
$n = 2^k$
Taking logarithm on both sides,
$k = \log_2 n$

Step 3: Substitute the value
$T(n) = T(1) + \log_2 n$
Since $T(1)$ is a constant,
$T(n) = \Theta(\log n)$

Save Answer Next →

QUESTION-9

Q9 **Substitution: Exponential Recurrence** 4 marks

Establish a tight asymptotic bound for T(n) = 2T(n−1) + 1 with T(0)=1. Explain the growth pattern.

Your Answer

Given recurrence relation:
T(n) = 2T(n - 1) + 1, where T(0) = 1.

Step 1: Expand the recurrence
T(n) = 2T(n - 1) + 1
T(n) = 2[2T(n - 2) + 1] + 1
T(n) = 4T(n - 2) + 2 + 1
T(n) = 8T(n - 3) + 4 + 2 + 1

After k steps,
T(n) = 2^k T(n - k) + (2^k - 1)

Step 2: Apply the base case
When k = n, T(n - k) = T(0) = 1.
Therefore,
T(n) = 2^n T(0) + (2^n - 1)
T(n) = 2^n + 2^n - 1
T(n) = 2^(n + 1) - 1

Step 3: Determine the asymptotic bound
The dominant term is 2^n.
Therefore, T(n) = Θ(2^n).

Save Answer Next →

QUESTION-10

Q10 **Check a Complexity Claim** 4 marks

An analyst claims T(n) = T(n/2) + n is Θ(n log n). Assess the claim and establish the correct asymptotic bound.

Your Answer

Given recurrence relation:
T(n) = T(n / 2) + n

The analyst claims that T(n) = Θ(n log n).
Step 1: Expand the recurrence
T(n) = T(n / 2) + n
T(n) = T(n / 4) + n / 2 + n
T(n) = T(n / 8) + n / 4 + n / 2 + n

Continuing until the base case,
T(n) = T(1) + 1 + 2 + 4 + ... + n

Step 2: Calculate the total work
The non-recursive work forms a geometric series:
T(n) = 1 + 2 + 4 + ... + n
Using the geometric series formula,
T(n) = 2n - 1

Step 3: Establish the correct asymptotic bound
The dominant term is n.
Therefore, T(n) = Θ(n).

Save Answer Next →

QUESTION-11

Q11 **Algorithm Crossover** 4 marks

Algorithm A performs $5n \log_2 n$ operations and Algorithm B performs $0.02n^2$ operations. Compare them and estimate the crossover input size.

Your Answer

Given:

Algorithm A: $A(n) = 5n \log_2 n$
Algorithm B: $B(n) = 0.02n^2$

Step 1: Find the crossover point

At the crossover point, both algorithms perform the same number of operations.
$5n \log_2 n = 0.02n^2$
Dividing both sides by n,
$5 \log_2 n = 0.02n$
$n = 250 \log_2 n$

Step 2: Estimate the input size
Solving $n = 250 \log_2 n$ numerically gives an approximate crossover point of $n \approx 3{,}000$.

Save Answer Next →

QUESTION-12

← Back

Q12 **Scalability: n log n** 4 marks

A program processes 10,000 records in 2 seconds using an $O(n \log_2 n)$ algorithm. Estimate runtime for 100,000 records, assuming proportional scaling and negligible fixed overhead.

Your Answer

Given:
Initial input size, $n_1 = 10{,}000$ records
Initial runtime, $T_1 = 2$ seconds
New input size, $n_2 = 100{,}000$ records
The algorithm has O(n log n) complexity.

Step 1: Apply the scaling formula
$T_2 / T_1 = (n_2 \log_2 n_2) / (n_1 \log_2 n_1)$

Step 2: Substitute the values
$T_2 = 2 \times (100{,}000 \log_2 100{,}000) / (10{,}000 \log_2 10{,}000)$
$T_2 = 2 \times 10 \times (\log_2 100{,}000 / \log_2 10{,}000)$
$T_2 \approx 2 \times 10 \times (16.61 / 13.29)$
$T_2 \approx 25$ seconds

Save Answer Next →

QUESTION-13

Q13 **Scalability: Quadratic** 4 marks

An $O(n^2)$ algorithm takes 4 seconds for 2,000 items. Estimate runtime for 6,000 items, assuming the quadratic term dominates.

Your Answer

Given:

Initial input size, $n_1 = 2{,}000$ items
Initial runtime, $T_1 = 4$ seconds
New input size, $n_2 = 6{,}000$ items
The algorithm has $O(n^2)$ time complexity.

Step 1: Apply the scaling formula
$T_2 / T_1 = (n_2 / n_1)^2$

Step 2: Substitute the values
$T_2 = 4 \times (6{,}000 / 2{,}000)^2$
$T_2 = 4 \times 3^2$
$T_2 = 4 \times 9$
$T_2 = 36$ seconds

Final Answer:

The estimated runtime for processing 6,000 items is 36 seconds, assuming the quadratic term dominates and fixed overhead is negligible.

Save Answer Next →

QUESTION-14

Q14 **Compare Three Growth Rates** 4 marks

Compare O(n), O(n log n), and $O(n^2)$ growth. Discuss how their operation counts scale and give an appropriate use case for each.

Your Answer

Comparison of O(n), O(n log n), and $O(n^2)$

1. O(n) – Linear Growth
In O(n) complexity, the number of operations increases directly with the input size. If the input size doubles, the operation count approximately doubles.
Use case: Finding a particular element by checking each item sequentially.

2. O(n log n) – Linearithmic Growth

In O(n log n) complexity, the number of operations increases slightly faster than linear growth. If the input size doubles, the operation count increases by slightly more than two times.
Example: Merge sort.
Use case: Sorting large datasets efficiently.

3. $O(n^2)$ – Quadratic Growth
In $O(n^2)$ complexity, the number of operations increases proportionally to the square of the input size. If the input size doubles, the operation count becomes approximately four times larger.
Example: Bubble sort.
Use case: Sorting small datasets or demonstrating simple sorting algorithms.
Comparison Table:

Complexity	Growth Pattern	If n Doubles	Example
O(n)	Linear	2 times	Linear search
O(n log n)	Linearithmic	Slightly more than 2 times	Merge sort
$O(n^2)$	Quadratic	4 times	Bubble sort

Save Answer Next →

QUESTION-15

Q15 **Constant Factors and Big-O** 4 marks

Two algorithms take $T_1(n)=100n$ and $T_2(n)=n \log_2 n$. Explain why asymptotic notation alone may not determine which is faster for small inputs.

Your Answer

Given:
Algorithm 1: $T_1(n) = 100n$
Algorithm 2: $T_2(n) = n \log_2(n)$

1. Asymptotic Comparison
Algorithm 1 has O(n) time complexity, while Algorithm 2 has O(n log n) time complexity.
As n increases, O(n) grows more slowly than O(n log n).

2. Effect of Constant Factors
Algorithm 1 has a large constant factor of 100, whereas Algorithm 2 has a smaller operation count for certain input sizes.
For small inputs, constant factors and implementation overhead can significantly affect actual execution time. Therefore, asymptotic notation alone cannot determine which algorithm is faster for small inputs.

3. Example
For n = 10:
$T_1(10) = 100 \times 10 = 1000$ operations
$T_2(10) = 10 \times \log_2(10) \approx 33.22$ operations
Here, Algorithm 2 performs fewer operations.

Save Answer Next →

QUESTION-16

Q16 **Hash Set: Time and Space** 4 marks

An algorithm scans an array once and stores distinct values in a hash set. Analyze expected time and auxiliary space, stating assumptions about hash operations.

Your Answer

Given:

An algorithm scans an array of n elements once and stores distinct values in a hash set.
1. Time Complexity
The algorithm scans each element exactly once. In a hash set, insertion and lookup take O(1) expected time, assuming a good hash function and a controlled load factor.
Therefore, processing n elements takes O(n) expected time.

2. Auxiliary Space Complexity
In the worst case, all n elements are distinct. The hash set stores all n elements, requiring O(n) auxiliary space.

3. Assumptions
- Hash operations take O(1) expected time.
- A good hash function distributes elements uniformly.
- Collisions are handled efficiently.
- The hash set resizes when necessary to maintain its load factor.

Final Answer:

Expected Time Complexity: O(n)
Auxiliary Space Complexity: O(n)

Save Answer Next →

QUESTION-17

Q17 **Recursive Temporary Arrays** 4 marks

A recursive algorithm splits an array into two halves, processes both halves, and allocates a temporary array of size n at each call. Analyze time and peak auxiliary space, clarifying whether allocations overlap.

Your Answer

Given:

A recursive algorithm divides an array into two halves, processes both halves, and allocates a temporary array of size n at each call.
1. Time Complexity
The recurrence relation is:
$T(n) = 2T(n/2) + O(n)$
Here, $a = 2$, $b = 2$, and $f(n) = O(n)$.
Using the Master Theorem:
$n^{(\log_2 2)} = n$
Since $f(n) = \Theta(n)$, Case 2 applies.
Therefore, the time complexity is $\Theta(n \log n)$.

2. Peak Auxiliary Space Complexity
At each recursive call, a temporary array proportional to the current subproblem size is allocated.
If both halves are processed sequentially and the temporary arrays are released before processing the next half, the peak auxiliary space is $O(n)$.
If both recursive calls execute simultaneously and their temporary arrays remain allocated, the peak auxiliary space can be $O(n \log n)$, because multiple levels of temporary arrays overlap.

3. Conclusion
Time Complexity: $\Theta(n \log n)$
Peak Auxiliary Space:
- $O(n)$, if recursive calls are processed sequentially and temporary arrays are released appropriately.

Save Answer Next →

QUESTION-18

Q18 **Nested Loops with a Condition** 4 marks

An algorithm has three loops from 1 to n, but the innermost statement executes only when $i+j \leq n$. Determine its time complexity and justify the count.

Your Answer

Given:

An algorithm contains three nested loops, each running from 1 to n. The innermost statement executes only when $i + j \leq n$.
Step 1: Count the iterations
The outer loop runs n times.
For each value of i, the second loop runs n times. However, the condition $i + j \leq n$ is satisfied only when $j \leq n - i$.
Therefore, the number of valid (i, j) pairs is:
$\Sigma(n - i)$, for $i = 1$ to n
$= (n - 1) + (n - 2) + \ldots + 1 + 0$
$= n(n - 1) / 2$
Thus, there are $\Theta(n^2)$ valid pairs.

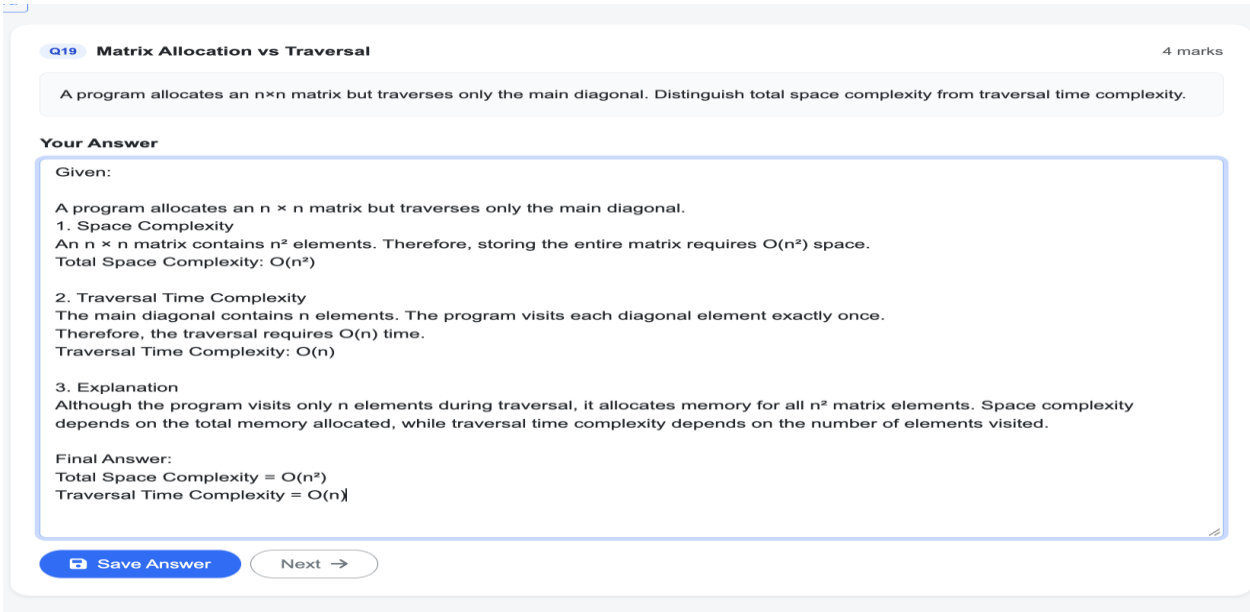
Step 2: Include the third loop
For each valid pair (i, j), the innermost loop runs up to n times.
Therefore, the total number of executions is:
$\Theta(n^2) \times \Theta(n) = \Theta(n^3)$

Step 3: Final complexity
The condition reduces the number of valid (i, j) pairs, but it does not change the overall cubic growth rate.

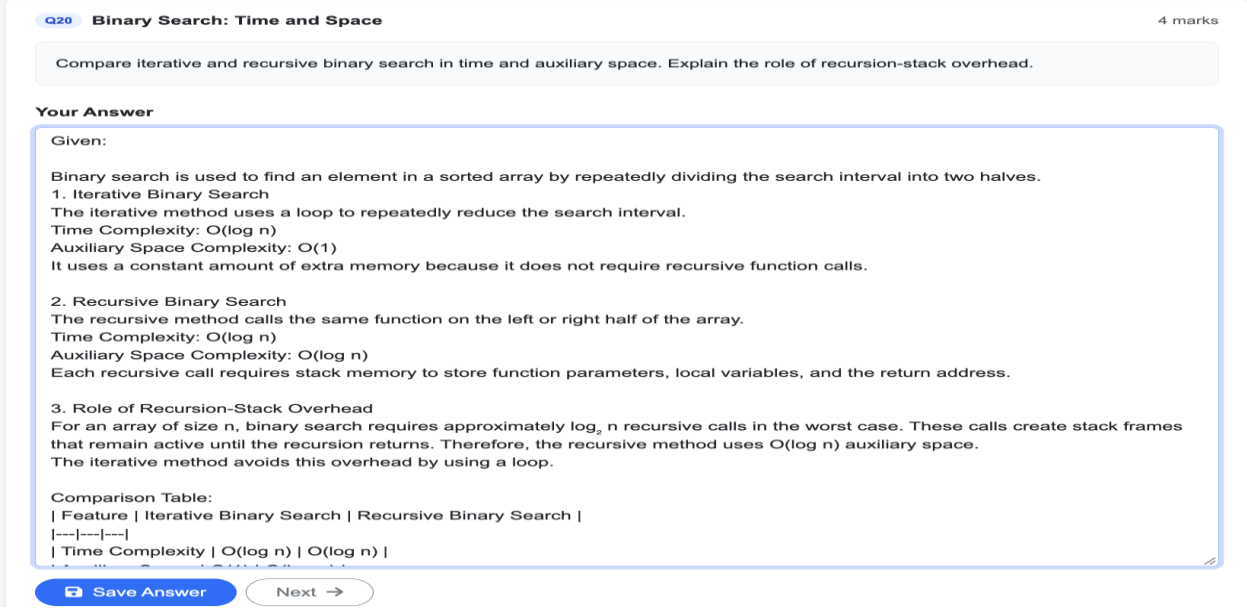
Final Answer:
Time Complexity: $\Theta(n^3)$
The innermost statement executes $\Theta(n^3)$ times because there are $\Theta(n^2)$ valid pairs and up to n iterations of the third loop for each

Save Answer Next →

QUESTION-19



QUESTION-20



QUESTION-21

Q21 **Unequal Subproblem Sizes** 4 marks

Analyze T(n)=T(n/3)+T(2n/3)+n using a recursion-tree argument. Explain the work per level and total tree depth.

Your Answer

Given recurrence relation:
T(n) = T(n/3) + T(2n/3) + n

Step 1: Work per Level
At the root level, the work is n.
The problem divides into two subproblems of sizes n/3 and 2n/3.
At the next level, the total non-recursive work is:
n/3 + 2n/3 = n
Similarly, at every level, the sum of the subproblem sizes is n. Therefore, the work per level is Θ(n).

Step 2: Total Tree Depth
The longest path follows the larger subproblem of size 2n/3.
After k levels, its size becomes:
n(2/3)^k
The recursion stops when:
n(2/3)^k = 1
Therefore, k = Θ(log n).
The shortest path also has logarithmic depth, so the recursion tree has Θ(log n) levels.

Step 3: Total Time Complexity
Each level performs Θ(n) work, and the tree has Θ(log n) levels.
Total work = Θ(n) × Θ(log n)
Final Answer:
T(n) = Θ(n log n)

Save Answer Next →

QUESTION-22

Q22 **Unequal Split with Linear Combine** 4 marks

A divide-and-conquer algorithm splits into subproblems of sizes n/4 and 3n/4 and performs Θ(n) combining work. Determine its asymptotic time complexity using a suitable method.

Your Answer

Given recurrence relation:
T(n) = T(n/4) + T(3n/4) + n

Step 1: Identify the subproblems
The algorithm divides the problem into two subproblems of sizes n/4 and 3n/4. The combining work is Θ(n).

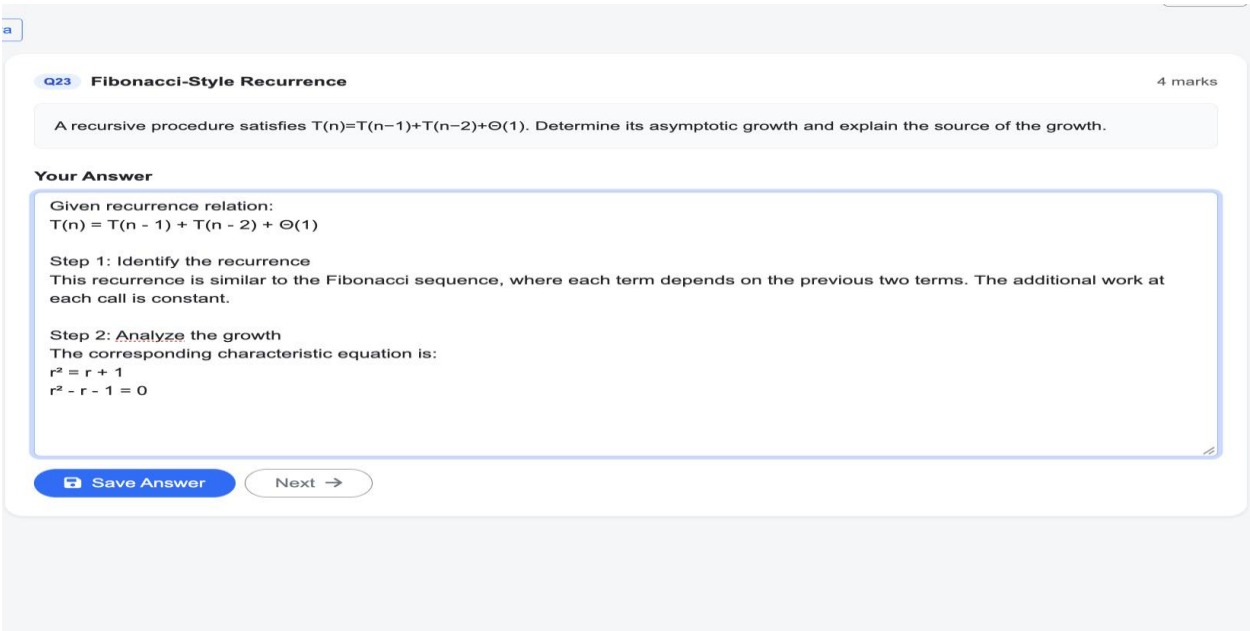
Step 2: Analyze the work per level
The total size of the subproblems is:
n/4 + 3n/4 = n
Therefore, the total non-recursive work at each level is Θ(n).

Step 3: Calculate the recursion depth
The longest path follows the subproblem of size 3n/4.
After k levels, its size is:
n(3/4)^k = 1
Taking logarithms gives:
k = Θ(log n)
Therefore, the recursion tree has Θ(log n) levels.

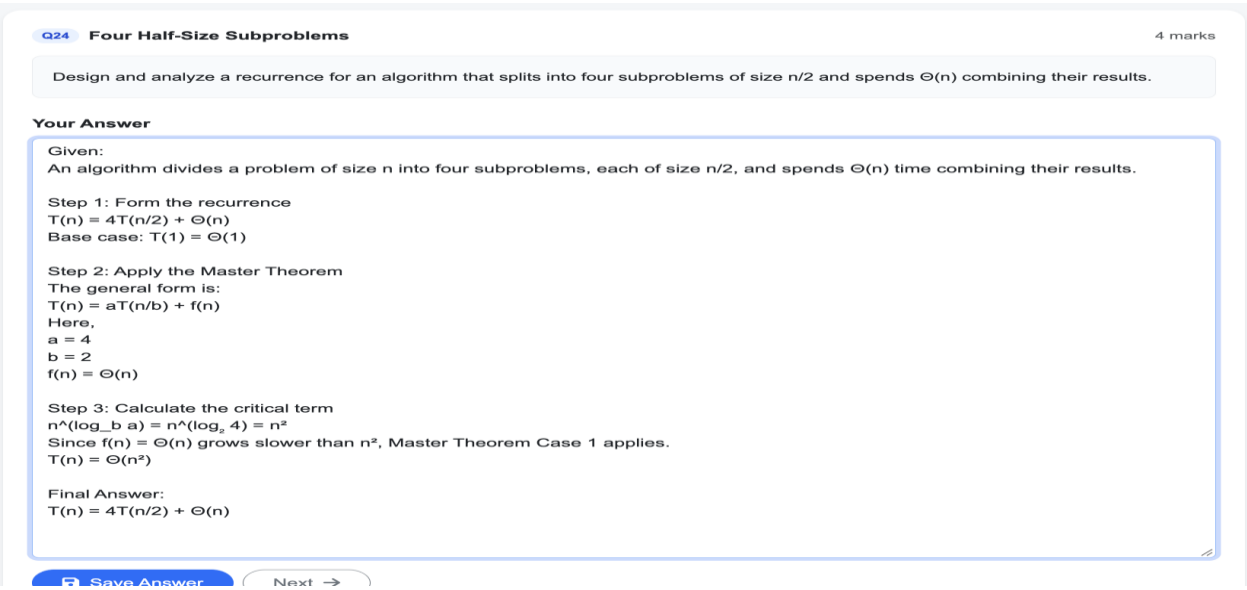
Step 4: Calculate the total time complexity
Each level performs Θ(n) work, and there are Θ(log n) levels.
T(n) = Θ(n) × Θ(log n)

Save Answer Next →

QUESTION-23



QUESTION-24



QUESTION-25

Q25 **Time-Space Trade-off** 4 marks

One algorithm uses O(n) time and O(n²) space; another uses O(n log n) time and O(n) space. Discuss which is more appropriate in a memory-constrained environment.

Your Answer

Given:

Algorithm A: O(n) time and O(n²) space.
Algorithm B: O(n log n) time and O(n) space.

1. Algorithm A
Algorithm A has linear time complexity, so it executes efficiently in terms of time. However, it requires quadratic space, which consumes a large amount of memory as the input size increases.

2. Algorithm B
Algorithm B has O(n log n) time complexity and O(n) space complexity. It takes slightly more time asymptotically than Algorithm A but requires significantly less memory.

3. Comparison
In a memory-constrained environment, Algorithm B is more appropriate because its space requirement grows linearly rather than quadratically.
For example, if the input size doubles, Algorithm A's space requirement increases approximately four times, whereas Algorithm B's space requirement increases approximately two times.

4. Conclusion
Algorithm B is the better choice in a memory-constrained environment because it uses O(n) auxiliary space instead of O(n²). Although Algorithm A may be faster, Algorithm B provides a better time-space trade-off when memory is limited.

Save Answer